

Softwaregestützte Modellbildung und Simulation: Exercise 1

Power method for finding dominant eigenvector of a matrix

Peleg Bar Sapir

June 1, 2026

1 Theoretical overview

1.1 Extracting the eigenvalue of a known eigenvector

Let $\vec{v} \in \mathbb{R}^n$ be an eigenvector of the matrix $\mathbf{A}$ with the corresponding eigenvalue λ . Then by normalizing $\vec{v}$ and multiplying it with $\mathbf{A}$ from both the right and the left, we can extract λ :

$$\begin{aligned}\hat{v}^T \mathbf{A} \hat{v} &= \hat{v}^T \lambda \hat{v} \\ &= \lambda \hat{v}^T \hat{v} \\ &= \lambda \|\hat{v}\|^2 \\ &= \lambda \cdot 1 \\ &= \lambda.\end{aligned}\tag{1}$$

1.2 The power method

The *dominant eigenvalue* λ_{dom} of a matrix $\mathbf{A}$ of dimensions $n \times n$ is the eigenvalue with largest (absolute) value:

$$\lambda_{\text{dom}} = \max |\lambda_i|,\tag{2}$$

where $\{\lambda_i\}$ are the eigenvalues of $\mathbf{A}$.

One can find the dominant eigenvector of $\mathbf{A}$ by using the *power method*:

1. let $\vec{v}_0 \in \mathbb{R}^n$ be a **non-zero** vector with random components.

2. Set $\lambda_0 = 1$ (note: the subscript denotes an iteration step, not the order of the eigenvalue/vector).
3. Apply $\mathbf{A}$ to $\vec{v}_0$ and normalize the result: $\vec{v}_1 = \mathbf{A}\vec{v}_0$, $\hat{v}_1 = \frac{1}{\|\vec{v}_1\|}\vec{v}_1$.
4. Extract λ_1 by multiplying $\mathbf{A}$ from the left by $\hat{v}_1^\top$ and from the right by $\hat{v}_1$ (Equation 1).
5. Repeat:
 - a) $\vec{v}_{i+1} = \mathbf{A}\hat{v}_i$,
 - b) $\hat{v}_{i+1} = \frac{1}{\|\vec{v}_{i+1}\|}\vec{v}_{i+1}$,
 - c) $\lambda_{i+1} = \hat{v}_{i+1}^\top \mathbf{A} \hat{v}_{i+1}$,
 until $|\lambda_i - \lambda_{i+1}| < \varepsilon$ (where $0 < \varepsilon \in \mathbb{R}$ is a pre-defined threshold).

2 Actual exercise

Implement the power method algorithm in Python, using NumPy:

1. Generate a random 5×5 matrix $\mathbf{A}$.
2. Generate a random starting vector $\vec{v} \neq \vec{0}$.
3. Implement the algorithm using the threshold $\varepsilon = 1.0 \times 10^{-7}$ and a maximum number of iterations $k_{\max} = 1\,000\,000$ (one million).
4. Compare your results to the standard NumPy function `numpy.linalg.eig()`.

Notes:

1. NumPy's `numpy.linalg.eig()` returns two variables: the eigenvalues of the input matrix as a single vector, and the corresponding eigenvectors as columns of a single matrix.
2. You can use the notation $\mathbf{A}@\mathbf{B}$ to calculate the product of two matrices $\mathbf{A}$ and $\mathbf{B}$.
3. You can use $\mathbf{A}.\mathbf{T}$ to get the transpose of the matrix $\mathbf{A}$.